

10. Fehlertoleranz

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

10.1 Vorschau

- Motivation
- Begriffe
- Ausfallerkennung
- CAP-Theorem
- Fehlermodelle
- Checkpointing & Recovery

10.1 Motivation

- Es gibt viele Fehlerquellen in der Software und der Hardware
- In verteilten Systemen:
 - Serverausfälle
 - Verlust von Netzwerkpaketen
 - Einfach oder mehrfache Netzwerkpartitionierungen
 - Unabsichtliche oder absichtliche falsche Antworten
- Serveranzahl & Fehlerhäufigkeit
 - Je mehr Server eingesetzt werden, desto höher ist die Wahrscheinlichkeit, dass einer davon ausfällt!

10.2 Begriffe

Allgemeine Terminologie

- Umfassender Begriff: **Verlässlichkeit** (engl. dependability)
- **Verfügbarkeit** (engl. availability): Wahrscheinlichkeit für das korrekte Arbeiten des Systems zu einem gegebenem Zeitpunkt.
- **Zuverlässigkeit** (engl. reliability): Zeitintervall solange das System sich korrekt verhält
 - Relevanter Begriff: Meantime between failure (MTBF).
- **Sicherheit** (engl. safety): Fehler hat keinen katastrophalen Effekt
 - Menschenleben nicht gefährdet, Datenbestand wird nicht zerstört, ...

Verfügbarkeit versus Zuverlässigkeit

- Beispiel 1: System fällt jede Stunde für eine Millisekunde aus
 - Entspricht einer Verfügbarkeit von 99,9999%, aber MTBF von knapp 1 Std.
- Beispiel 2: System fällt jedes Jahr für einen Monat aus
 - Verfügbarkeit von 91,67%, aber MTBF von einem Jahr.

Fehlerbegriffe

- **Fehlerursache (engl. fault):** Ursache für einen eingetretenen Fehler
 - z.B. einmalige sporadisch oder permanente Störung
- **Eingetretener Fehler (engl. error):** Teil eines unerlaubten Systemzustands, der zu einem Ausfall führen kann (falls nicht entsprechend behandelt).
- **Ausfall (engl. failure):** System verhält sich nicht spezifikationskonform
 - Im besten Fall hält das System an

Fehlerbegriffe: Beispiel

- **Fault:** Ein Bitfehler bei der Datenübertragung
 - z.B. DNS-Anfrage zur Auflösung einer IP-Adresse
- **Error:** Das führt zu falsch interpretierten Empfangsdaten
 - z.B. falsche IP-Adresse
- **Failure:** führt zu Unfähigkeit zur Kommunikation
 - z.B. Anwendungskomponente nicht erreichbar

10.3 Ausfallerkennung

Ausfallerkennung

- Hier geht es darum, dass erkannt wird, dass ein Server ausgefallen ist.
- Wichtig: Es gibt keine perfekte Ausfallerkennung in einem verteilten System, weil es keine Obergrenze für die Laufzeit von Netzwerkpaketen gibt
- Problem bei beiden Ansätzen: Wahl des Timeouts
 - Zu klein: Server werden fälschlicherweise als ausgefallen erkannt
 - Zu groß: es dauert lange, bis ein Recovery eingeleitet wird
 - Der passende Wert hängt stark von der Netzwerkkumgebung ab

- Lösung 1: Heart-Beat-Nachrichten
 - Es werden periodisch Heart-Beat-Nachrichten verschickt, die vom Empfänger bestätigt werden
 - Als Optimierung werden die Bestätigungen mit anderen Antworten mitgeschickt
→ engl. piggybacked
 - Bleibt die Antwort über einen definierten Zeitraum (engl. timeout) aus, so geht der anfragende Server davon aus, dass der Empfänger ausgefallen ist.

- Lösung 2: Sperr-Dateien, verwaltet durch einen Koordinator
 - Es gibt einen zentralen Koordinator (z.B. Zookeeper), welcher kleine Dateien speichert und Sperren darauf verwaltet (siehe später)
 - Jeder Server erzeugt eine Datei auf dem Koordinator und sperrt diese
 - Die Sperre gilt nur für eine bestimmte Zeit und muss periodisch durch den Server erneuert werden
 - Der Koordinator versucht periodisch diese Sperr-Dateien zu sperren
 - Sobald dies klappt erkennt der Koordinator den Timeout des Servers
 - Aber auch der Server erkennt die Situation, falls er verspätet versucht die Sperre zu erneuern und diese nicht mehr bekommt
- Offensichtlich kann es bei beiden Lösungen passieren, dass ein Server nach dem Timeout antwortet, da er fälschlicherweise als ausgefallen erkannt wurde.

10.4 Fehlermodelle

Crash Failure

- **Crash Failure:** Rechner- bzw. Programmabsturz
 - Z.B. Server stürzt ab und beantwortet keine Anfragen mehr
→ Fail-Stop-Verhalten
 - Problem: Absturz und langsames Antworten schlecht unterscheidbar.
- Eignet sich für „gutmütige“ Fehler: Ein fehlerhafter Prozess führt keine Aktionen aus, die ein korrekter Prozess nicht machen würde.
- Wird häufig innerhalb eines Data-Centers verwendet
 - Vertrauenswürdige Umgebung (engl. trusted environment)

Byzantine Failure

- **Byzantine Failure:** byzantinische Fehler
 - System kann sich beliebig falsch verhalten,
 - Z.B. Server schickt gelegentlich falsche Antworten und erreicht gelegentlich falsche Systemzustände,
- Eignet sich für bösartige Fehler: Fehlerhafte Prozesse können beliebige Aktionen ausführen und dabei auch untereinander kooperieren oder sogar absichtlich das System stören
- Notwendig für verteilte Anwendungen im Internet
 - Skaliert aber sehr schlecht (siehe Kapitel Konsensus)

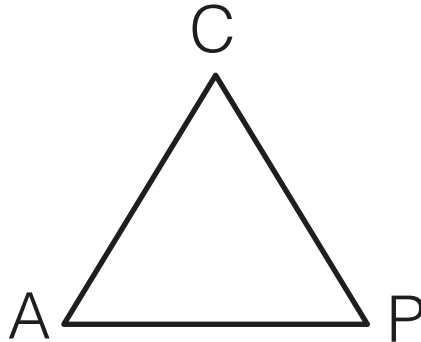
10.5 CAP-Theorem

CAP-Theorem

- Das CAP-Theorem bezieht sich auf eine verteilte Anwendung oder ein verteiltes System, welches auf verteilt replizierten Daten arbeitet
 - Das ist im Prinzip gegeben
- Das Theorem sagt, dass es in einem verteilten System unmöglich ist, gleichzeitig Konsistenz, Verfügbarkeit und Partitionstoleranz zu garantieren
→ es sind immer nur maximal zwei dieser Eigenschaften erzielbar

CAP-Theorem

- **Konsistenz (engl. Consistency):**
alle Server sehen beim Lesen immer den zuletzt geschriebenen Wert
- **Verfügbarkeit (engl. Availability):**
Alle Anfragen an das System werden in endlicher Zeit beantwortet
- **Partitionstoleranz (engl. Partition Tolerance):**
Das System arbeitet auch bei Netzwerkpartitionierung weiter.



Beispiele

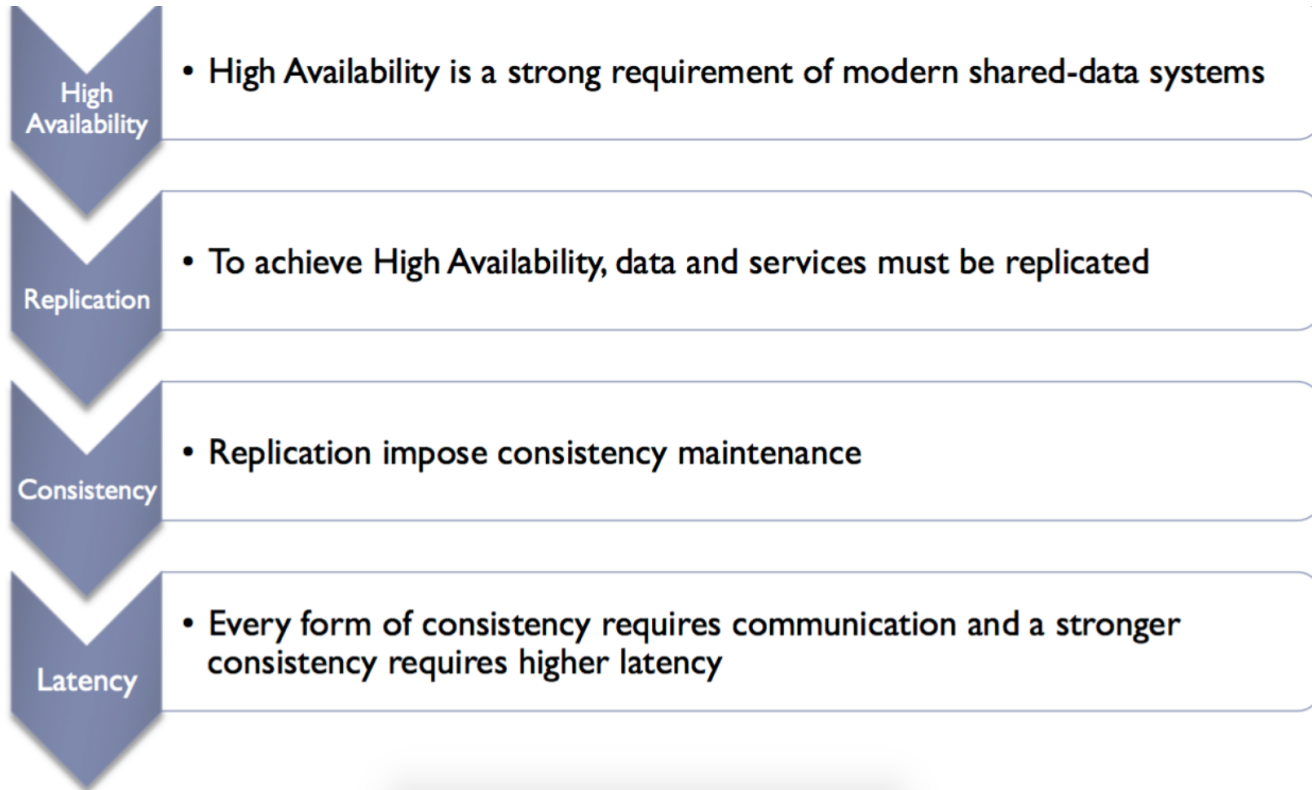
- CP: quorum-basierte Replikation
 - Garantiert Konsistenz und toleriert Netzwerkpartitionierung
 - System ist aber bei einer Partitionierung nicht für alle Server verfügbar
- AP: Domain Name System
 - Bleibt auch bei Netzwerkpartitionen verfügbar, verzichtet aber strenge Konsistenz
- CA: Relationale Datenbank (RDBMS)
 - Wenn diese nur auf einem Server ausgeführt werden
 - Dann sind keine Maßnahmen für Netzwerkpartitionen notwendig, aber es entsteht ein Single Point of Failure!

Konsequenzen für die Praxis

- Verteilte Systeme müssen immer Netzwerk-Partitionierungen berücksichtigen
 - Diese können auch innerhalb eines Data-Centers auftreten, wenn auch selten
- Somit beschränkt sich die Wahl auf Konsistenz und Verfügbarkeit
 - Wichtig: dies ist keine binäre Entscheidung
 - Beispiel: Flug-Reservierungssystem
 - Wenn viele Plätze frei sind, reicht eine schwache Konsistenz
 - Wenn nur noch wenige Sitzplätze frei sind, wird eine strenge Konsistenz benötigt, um Überbuchungen zu vermeiden



Konsistenz vs. Latenz



Warum ist das CAP-Theorem wichtig?

- Daten-Verarbeitung/-Verwaltung ist fast immer verteilt!
- CAP-Theorem beschreibt „Tradeoffs“ bzgl. Replikation in verteilten Systemen
- Ursprüngliche Rede von E. Brewer im Jahr 2000.
 - Principle of Distributed Computing Conference
- Der Beweis seiner Aussage erfolgte 2002 durch Gilbert Lynch
 - “Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services.”, S. Gilbert and N. Lynch, ACM SIGACT News, Vol. 33, Issue 2, 2002.

10.6 Checkpointing & Recovery

- Ziel: fehlerhaften Zustand in einen korrekten überführen und vermeiden, dass das System auf den initialen Zustand zurückfällt
- Wichtig für lang laufende Anwendungen
 - Wissenschaftliche Berechnungen/Simulationen → High Performance Computing
 - Soziale Netzwerke (für Graph-Daten)
 - Online Computer-Spiele
 - ...

- Vorwärtsbehebung (engl. forward recovery):
 - Versetzt System in konsistenten Zustand, ohne Rücksetzung in einen alten Zustand
 - Problem: für jede Fehlersituation wird individuelle Aktion benötigt
- Rückwärtsbehebung (engl. backward recovery):
 - Periodisch Sicherungspunkte (engl. checkpoints) speichern
 - Bei einem Fehler wird auf den letzten Checkpoint zurückgesetzt und von dort aus weitergerechnet
 - Checkpointing und Recovery sind u.U. aufwendig

Beispiel Empfang einer defekten Nachricht

- Forward Recovery:
 - Redundanz in der Nachricht erlaubt Fehlerkorrektur
 - Die Nachricht kann also lokal repariert werden
 - Forward Error Correction (FEC), z.B. Parity, Hamming Code, ...
- Backward Recovery:
 - Retransmission beim Sender anfordern

Zustandssicherung beim Checkpointing

- Umfang der Zustandssicherung:
 - Es genügt nicht, nur den Speicher der Anwendung zu sichern
 - Wichtig sind auch Betriebssystemzustände
 - Sowie Gerätezustände (z.B. Dateiinhalt, Kommunikationskanäle?, ...)
- Transparenz der Zustandssicherung:
 - Werden alle relevanten Zustände transparent gesichert?
 - Oder muss die Anwendung dies machen = application-level checkpointing
 - Hierzu registriert die Anwendung eine Callback-Routine
 - Diese ruft die CP-Implementierung beim Checkpoint & Restart auf

Speicherung der Checkpoints

- Checkpoints werden i.d.R. in persistentem Speicher abgelegt
 - z.B. RAID-Disk, verteiltes Dateisystem
- Schreiben auf Disk ist vergleichsweise langsam:
 - Inkrementelle Datensicherung oder Kompression (CPU-Aufwand)
 - Verteilt (viele Disks) oder asynchron Schreiben (zuerst Puffern)
- Bem.: Diskspeicher muss irgendwann reorganisiert werden:
 - Sicherungspunkte müssen gelöscht werden
 - Aber auch alte Checkpoints notwendig, da evt. unentdeckte Fehler.

Rücksetzbarkeit

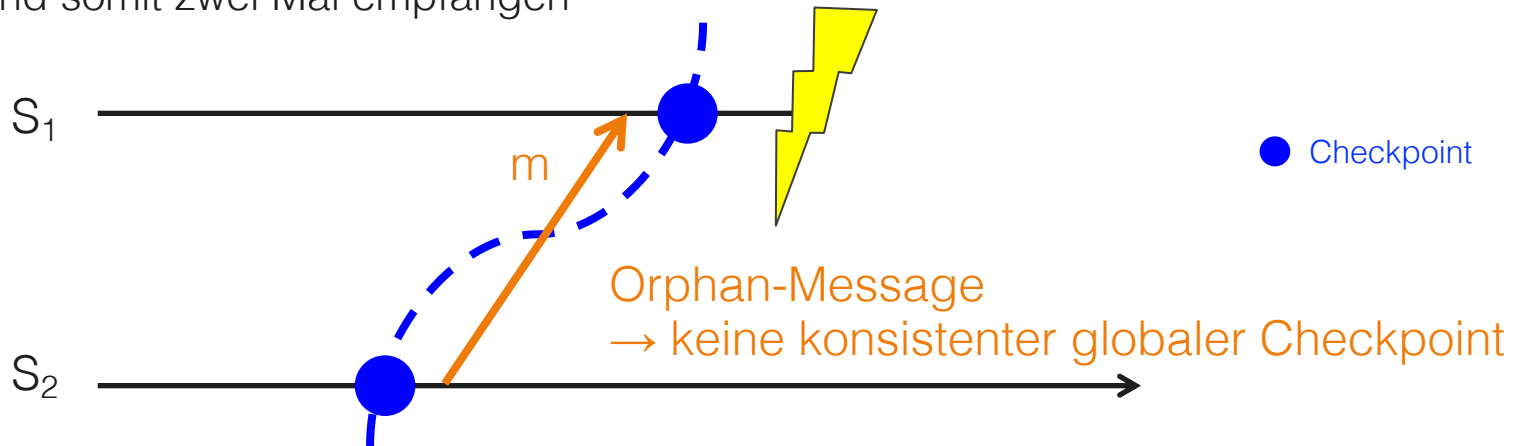
- Speicher: einfach vor der Änderung eine Schattenkopie anlegen
- Erhaltene Sperren müssen zurückgefordert werden
- Eine Hardware-Ausgabe ist u.U. nicht rücksetzbar, ...
 - bereits gesendete Nachricht kann nicht zurückgefordert werden, ...

Verteiltes Checkpointing

- Im verteilten Fall speichert jeder Server der Anwendung periodisch lokale Checkpoints
- Im Fehlerfall werden alle Server auf die Menge der letzten lokalen Checkpoints (=globaler Checkpoint) zurückgesetzt und starten von dort aus das Recovery
- Problem: evt. ist die Menge der letzten lokalen Checkpoints nicht konsistent und es muss erst eine geeignete Kombination von lokalen Checkpoints gefunden werden, welche konsistent ist

Konsistente globale Checkpoints

- = dürfen keine Orphan-Messages enthalten
 - Lokaler Checkpoint von S_1 enthält Nachrichtempfang, jedoch ist das Sendereignis nicht im lokalen Checkpoint von S_2 enthalten
→ kausaltreues Beobachten verletzt
 - Bei einer Rücksetzung im Fehlerfall würde die Nachricht erneut gesendet und somit zwei Mal empfangen

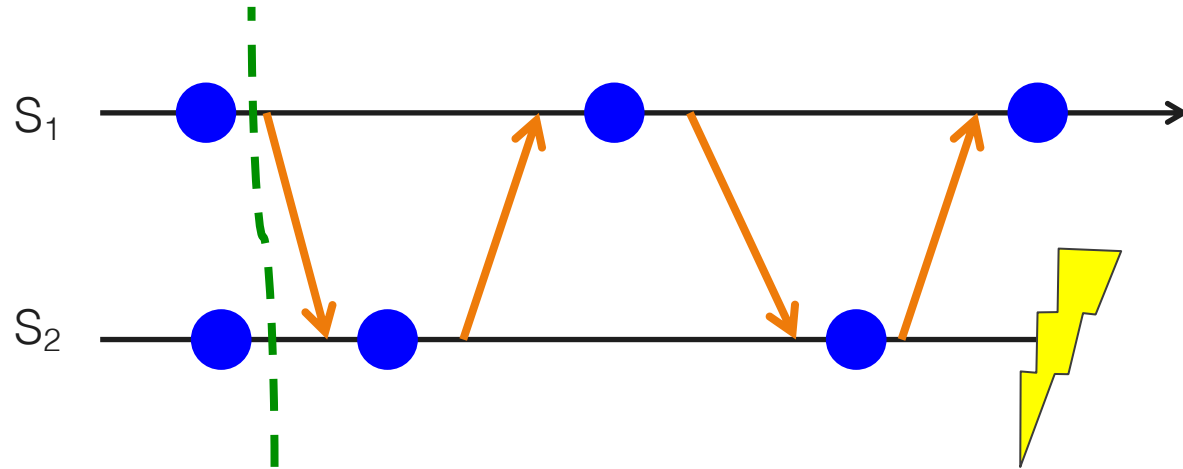


Unabhängiges Checkpointing

- Engl. uncoordinated / independent checkpointing
- Jeder Server sichert unabhängig von anderen lokale Checkpoints
- Keine „Kosten“ durch Koordination von Servern
- Aber u.U. bildet die Menge der letzten lokalen Checkpoints keinen konsistenten Zustand:
 - Im Fehlerfall muss ein konsistenter Zustand berechnet werden
→ „recovery line computation“
 - Hierzu werden evt. auch ältere lokale Checkpoints benötigt
 - Jeder Server muss somit u.U. viele lokale Checkpoints speichern

Unabhängiges Checkpointing – Domino Effekt

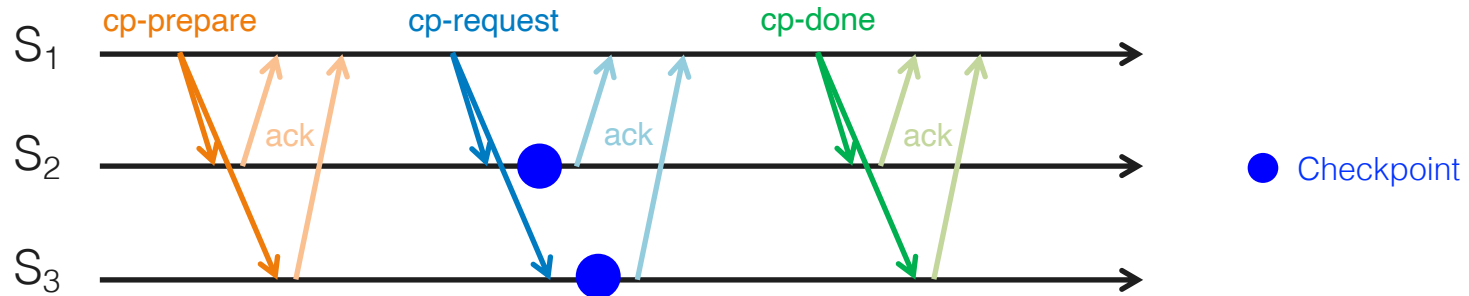
- Im worst-case fällt das System durch den Domino-Effekt auf den Initialzustand zurück
- Beispiel:



Recovery-Line (unmittelbar davor ist der jüngste global konsistenter Checkpoint)

Koordiniertes Checkpointing

- System anhalten, bis jeder Server seinen lokalen Checkpoint geschrieben hat
→ Domino-Effekt wird vermieden (da keine Orphan Messages möglich sind)
- Einfache Lösung mit 3 Phasen:



Koordiniertes Checkpointing

- Nach dem Empfang von „cp-prepare“ stoppt der Server und verschickt ,keine Anwendungsnachrichten mehr.
- Nachrichten die nach dem Stoppen, also nach dem Empfang von „cp-prepare“ eintreffen, werden lokal gepuffert
 - Sie sind nicht Teil des lokalen Checkpoints
 - Werden nach dem Empfang von „cp-done“ ausgeliefert
 - Damit werden Orphan Messages vermieden

Koordiniertes Checkpointing - Bewertung

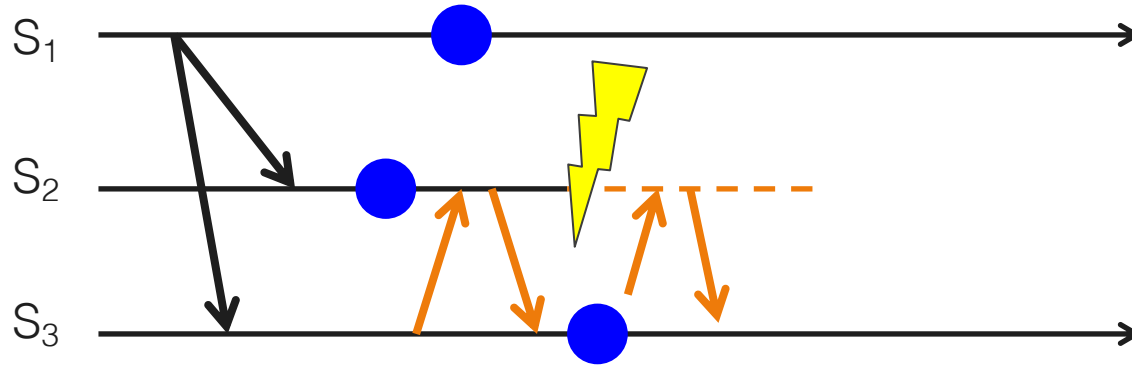
- Positiv:
 - Relativ einfach zu implementieren, es muss immer nur „ein“ lokaler Checkpoint gespeichert werden
 - Zwischen zwei lokalen CPs fällt im normalen Betrieb kein weiterer Aufwand an
- Negativ:
 - Checkpoint-Intervall ist in der Praxis oft groß, z.B. 15-120min.
 - Erheblicher Zeitaufwand für die Koordinierung aller Server, vor allem bei einer großen Anzahl an Servern

Nachrichtenprotokollierung

- Idee: protokolliere Nachrichten (engl. logging) auf Disk
 - entweder beim Sender oder Empfänger
- Recovery:
 - Nur betroffene Server werden auf ihren letzten lokalen Checkpoint zurückgesetzt und starten von dort aus neu
 - Mit Hilfe der Logdatei werden Nachrichten gesendet und empfangen
 - Gesendete Duplikate werden mit Sequenznummer gefiltert
 - Nachrichtenempfang wird durch Logdatei bedient
- Funktioniert nur bei deterministischen Abläufen!

Nachrichtenprotokollierung - Beispiel

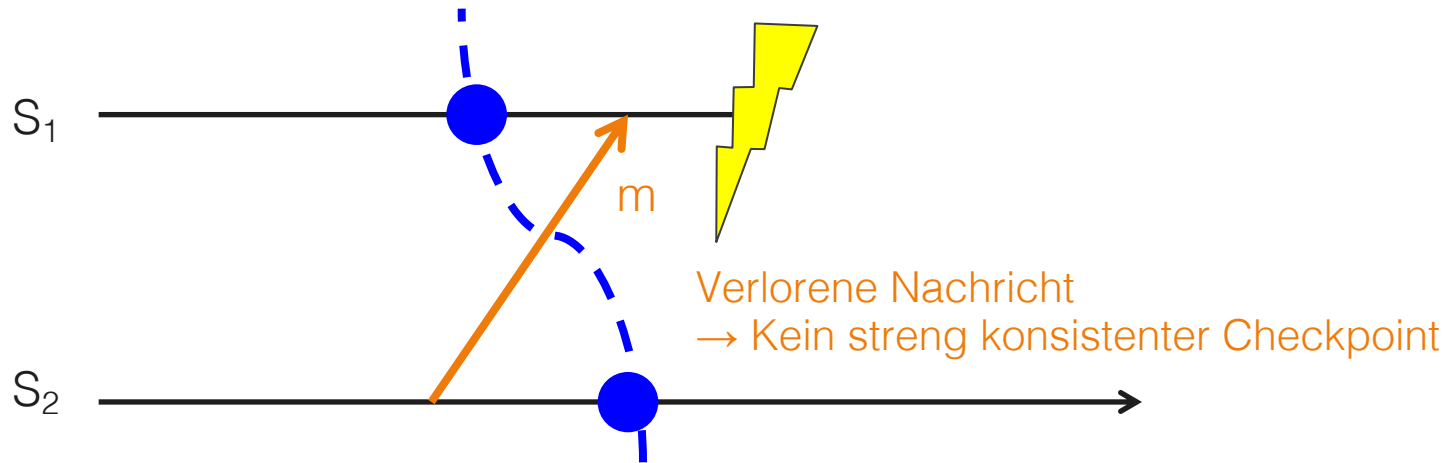
- Beispiel: Protokollierung beim Empfänger
 - Beim Recovery von S_2 **werden orange Nachrichten aus der Logdatei gelesen**
 - S_3 filtert doppelt empfangene Nachricht



- Vorteile:
 - Generell wird eine globale Rücksetzung vermieden
 - Und trotz unabhängigem Checkpointing reicht pro Server ein lokaler Checkpoint

Strenge Konsistenz globaler Checkpoints

- = Vermeidung von Orphan Messages und zusätzlich Vermeidung von verlorenen Nachrichten (engl. lost messages)
 - Lokaler Checkpoint von S_2 hat das Sendeereignis von m gespeichert
 - Lokaler Checkpoint von S_1 hat jedoch m nicht gesichert



Strenge Konsistenz globaler Checkpoints

- Entweder durch Logging von Nachrichten, um Server einzeln wiederherzustellen
- Oder Spülen von Nachrichten aus den Kanälen
 - Synchron (DMTCP, XtremGCP)
 - Asynchron (Chandy & Lamport)

10.8 Zusammenfassung

Zusammenfassung

- Es gibt keine perfekte Ausfallerkennung
- Fehlermodelle unterscheiden gutartige und bösartige Fehler
- CAP-Theorem sagt, dass bei einem verteilten System ein Kompromiss zwischen Verfügbarkeit und Konsistenz eingegangen werden muss.
- Checkpointing & Recovery ist sinnvoll für lang laufende Anwendungen
- Verteilte Checkpoints sind
 - konsistent, wenn beim Recovery keine Orphan Message auftritt
 - Streng konsistent, wenn beim Recovery keine Orphan und keine Lost Message auftritt